
title: "OPTI-X GALAXY UG — Beyond The Stage" subtitle: "Full Technical & Functional Documentation" date: "September 2026"

OPTI-X GALAXY UG

Beyond The Stage

UG Business Analytics Game 2026 — Full Technical & Functional Documentation

Version 0.1.0 · Last Updated: September 14, 2026

Table of Contents

- 1. [Project Overview](#)
- 2. [Tech Stack](#)
- 3. [Architecture Overview](#)
- 4. [Database Schema](#)
- 5. [Authentication & Authorization](#)
- 6. [API Reference](#)
- 7. [Frontend Pages & User Flow](#)
- 8. [Components Library](#)
- 9. [Game Mechanics & Scoring](#)
- 10. [Admin Dashboard](#)
- 11. [Types & Utilities](#)
- 12. [Configuration & Environment](#)
- 13. [Deployment Guide](#)
- 14. [Project Structure](#)

1. Project Overview

1.1 Introduction

OPTI-X GALAXY UG (codenamed *Beyond The Stage*) is a real-time, multiplayer business analytics simulation game designed for undergraduate students. Built for the **Analytics Konclave 2026** at **KIIT School of Management (KSOM)**, it challenges teams to forecast concert/event attendance by analyzing 60 historical event records and predicting General Admission (GA) and VIP/Premium occupancy across 6 competitive rounds.

1.2 Game Concept

Teams take on the role of **Event Analytics Consultants** advising a concert promotion company. They must study historical concert data — including artist prestige, venue type, weather conditions, promotional intensity, competition, and more — to predict audience turnout for future events. The game tests analytical thinking, pattern recognition, and forecasting accuracy under timed conditions.

1.3 Key Features

Feature	Description
60 Historical Events	Rich training dataset with 15+ demand variables
6 Live Rounds	Each round presents a unique scenario with new event parameters
Real-Time Timer	Configurable countdown timer (default 10 minutes per round)
Dual Forecasting	Teams predict both GA (0–50,000) and VIP (0–10,000) occupancy
Weighted Scoring	60% GA accuracy + 40% VIP accuracy, with penalty modifiers
Live Leaderboard	Real-time competitive rankings
Admin Command Center	Full game master control panel
Session Conflict Handling	Prevents multi-device login with force-login option
Bulk Team Import	Excel-based team onboarding
Export Capabilities	Export teams and submissions to Excel

2. Tech Stack

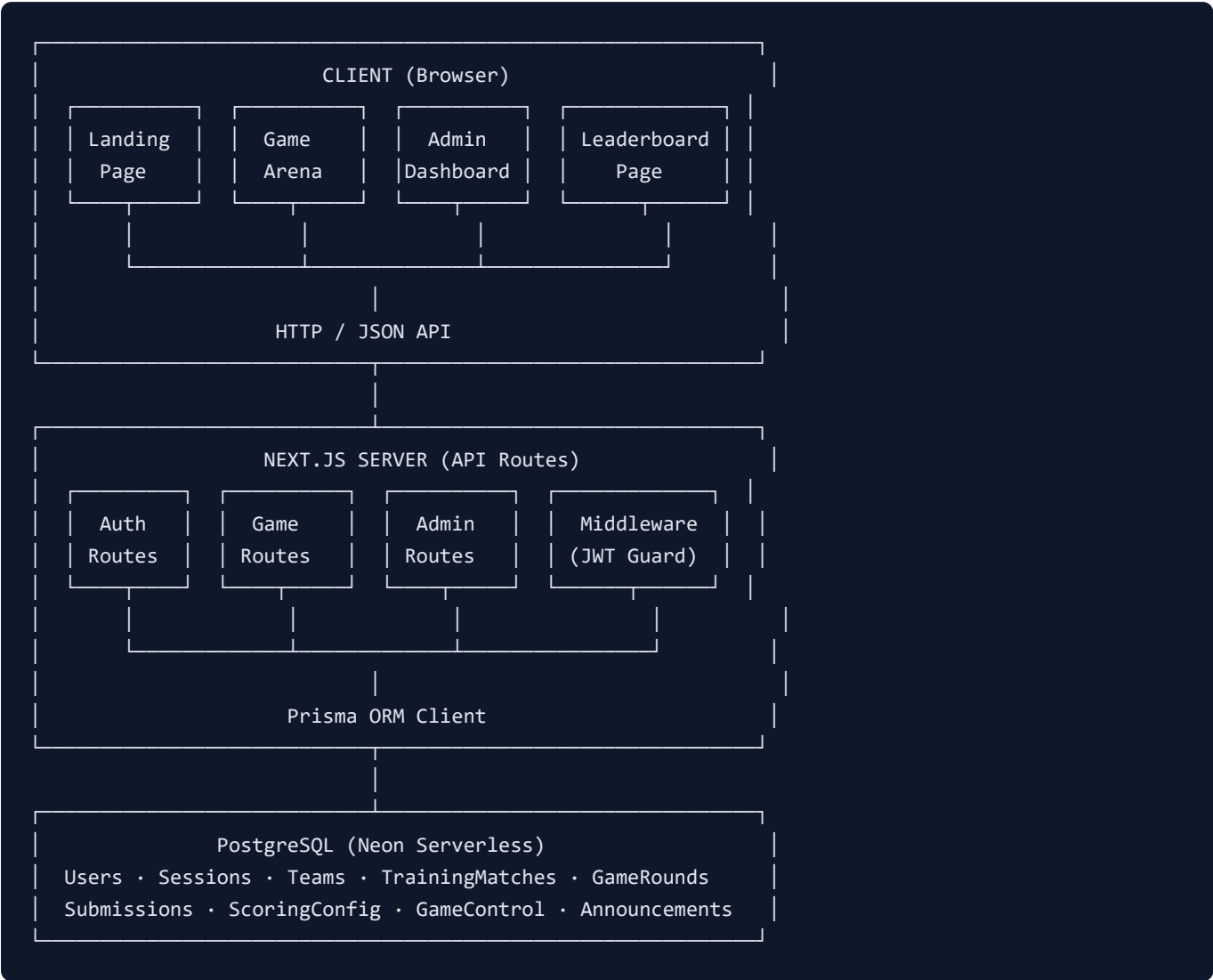
2.1 Core Technologies

Layer	Technology	Version
Framework	Next.js (App Router)	14.2.35
Language	TypeScript	5.x
UI Library	React	18.x
Styling	Tailwind CSS	3.x
Animations	Framer Motion	—
Icons	Lucide React	—
Charts	Recharts	—
Database	PostgreSQL (Neon Serverless)	—
ORM	Prisma	6.19.3
Auth (JWT)	jose	—
Password Hashing	bcryptjs	—
Excel I/O	xlsx (SheetJS)	—

2.2 Development Tools

Tool	Purpose
ESLint	Code linting
PostCSS	CSS processing pipeline
Prisma Studio	Database GUI (<code>npm run db:studio</code>)

3. Architecture Overview



4. Database Schema

4.1 Entity Relationship Overview

The database consists of **10 core models** managed via Prisma ORM.

4.2 Model Definitions

4.2.1 User (Admin Accounts)

Field	Type	Description
id	String (cuid)	Primary key
email	String (unique)	Admin email address
passwordHash	String	bcrypt-hashed password
role	String (default: "ADMIN")	User role
createdAt	DateTime	Account creation timestamp

4.2.2 Session

Field	Type	Description
id	String (cuid)	Primary key
userId	String	FK → User
tokenHash	String	Hashed session token
expiresAt	DateTime	Session expiry
isActive	Boolean	Active session flag

4.2.3 Team (Player Teams)

Field	Type	Description
id	String (cuid)	Primary key
teamCode	String (unique)	Team identifier (e.g., "TEAM-01")
name	String	Display name
pin	String	bcrypt-hashed access PIN
rawPin	String?	Plaintext PIN (for admin visibility)
currentRound	Int (default: 1)	Current round the team is on
totalScore	Float (default: 0)	Cumulative score
status	String (default: "active")	Team status
institution	String?	Institution name
sessionToken	String?	Active session token
sessionStartedAt	DateTime?	When the current session started
roundStartedAt	DateTime?	When the current round timer started
createdAt	DateTime	Registration timestamp

Relations: submissions → Submission[]

4.2.4 TrainingMatch (60 Historical Events)

Field	Type	Description
id	Int (autoincrement)	Primary key
eventId	String (unique)	Event identifier (T001–T060)
artistAct	String	Artist/act performing
venueType	String	Venue type (Arena, Stadium, etc.)
dayOfWeek	String	Day of the week
isWeekend	Int	Weekend flag (0/1)
isHoliday	Int	Holiday flag (0/1)
weatherCondition	String	Weather condition
temperature	Float	Temperature reading
promoLevel	String	Promotional intensity level
promoScore	Float	Promotional score index
artistPrestigeIndex	Float	Artist prestige rating
competitorEvents	Int	Concurrent competing events
socialBuzz	Float	Social media buzz index
ticketPriceGA	Float	GA ticket price
ticketPriceVIP	Float	VIP ticket price
historicalAvgOccupancy	Float	Historical average occupancy
daysBeforeSoldOut	Float	Days before event sold out
actualGa	Int	Actual GA attendance (answer)
actualVip	Int	Actual VIP attendance (answer)

4.2.5 GameRound (6 Live Rounds)

Field	Type	Description
<code>id</code>	Int (autoincrement)	Primary key
<code>roundNumber</code>	Int (unique)	Round 1–6
<code>title</code>	String	Round title (e.g., "Monsoon Impact")
<code>theme</code>	String	Thematic scenario
<code>instructions</code>	String	Scenario description text
All event variables	Various	Same variables as TrainingMatch
<code>actualGa</code>	Int	Correct GA answer
<code>actualVip</code>	Int	Correct VIP answer
<code>status</code>	String (default: "ready")	Round status
<code>timerSeconds</code>	Int (default: 600)	Timer duration per round

4.2.6 Submission

Field	Type	Description
<code>id</code>	String (cuid)	Primary key
<code>teamId</code>	String	FK → Team
<code>roundNumber</code>	Int	Which round (1–6)
<code>predictedGa</code>	Int	Team's GA prediction
<code>predictedVip</code>	Int	Team's VIP prediction
<code>reasoning</code>	String?	Team's analytical reasoning
<code>gaError</code>	Float	Absolute error for GA
<code>vipError</code>	Float	Absolute error for VIP
<code>accuracy</code>	Float	Combined accuracy score
<code>score</code>	Float	Final round score
<code>submittedAt</code>	DateTime	Submission timestamp

Constraints: Unique on `[teamId, roundNumber]`

4.2.7 ScoringConfig

Field	Type	Default	Description
<code>gaWeight</code>	Float	0.6	Weight for GA accuracy (60%)
<code>vipWeight</code>	Float	0.4	Weight for VIP accuracy (40%)
<code>maxRoundScore</code>	Float	1000	Maximum score per round
<code>overForecastPenalty</code>	Float	1.0	Penalty multiplier for over-forecasting
<code>underForecastPenalty</code>	Float	1.0	Penalty multiplier for under-forecasting

4.2.8 GameControl

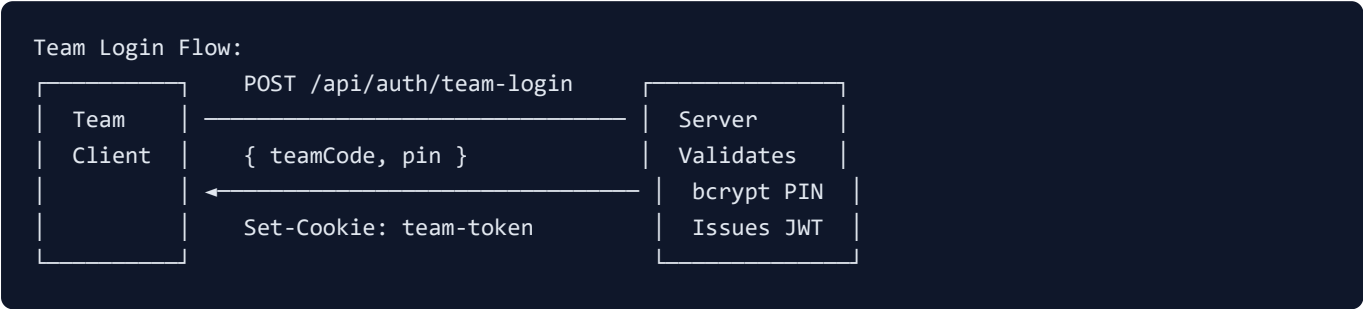
Field	Type	Default	Description
<code>gameStatus</code>	String	"lobby"	Current game state
<code>activeRound</code>	Int	1	Active round number
<code>timerEnabled</code>	Boolean	true	Whether timer is active
<code>leaderboardEnabled</code>	Boolean	true	Whether leaderboard is visible
<code>timerStartedAt</code>	DateTime?	—	When the timer was started
<code>roundTimerSeconds</code>	Int	600	Timer duration

4.2.9 Announcement & AuditLog

Standard models for system-wide broadcast messages and action logging respectively.

5. Authentication & Authorization

5.1 Team Authentication



- **Endpoint:** POST /api/auth/team-login
- **Input:** teamCode (string) + pin (string)
- **Validation:** PIN verified against bcrypt hash in database
- **Session:** Sets team-token HTTP-only cookie (12-hour expiry)
- **Conflict Handling:** If a team is already logged in elsewhere, returns SESSION_CONFLICT error. Client can send forceLogin: true to override.

5.2 Admin Authentication

- **Endpoint:** POST /api/auth/login
- **Input:** email (string) + password (string)
- **Validation:** Password verified against bcrypt hash
- **Session:** Generates a JWT via jose containing { userId, role: 'ADMIN' }, stored in oxg-session HTTP-only cookie (24-hour expiry)

5.3 Middleware Protection

The middleware at src/middleware.ts intercepts all requests:

Route Pattern	Protection
/game/*	Requires valid team-token cookie
/admin/*	Requires valid oxg-session JWT with role === 'ADMIN'
All other routes	Public access

6. API Reference

6.1 Authentication Endpoints

POST /api/auth/login

Purpose: Admin login

Body: { email: string, password: string }

Response: { success: true } + sets oxg-session cookie

Errors: 401 Invalid credentials

POST /api/auth/team-login

Purpose: Team login

Body: { teamCode: string, pin: string, forceLogin?: boolean }

Response: { success: true, teamId, teamCode, name } + sets team-token cookie

Errors: 401 Invalid credentials, 409 SESSION_CONFLICT, 403 GAME_COMPLETED

POST /api/auth/logout / POST /api/auth/team-logout

Purpose: Clear session cookies

Response: { success: true }

GET /api/auth/me

Purpose: Get current authenticated user/team info

Response: { type: 'admin'|'team', data: {...} }

POST /api/auth/register

Purpose: Register new team or admin account

Body: { teamCode, name, pin, institution? } or { email, password }

6.2 Game Endpoints

GET /api/game/status

Purpose: Get current game state

Auth: team-token

Response:

```
{
  "currentRound": 1,
  "status": "active",
  "timerState": {
    "timerStarted": true,
    "remaining": 540,
    "total": 600
  }
}
```

GET /api/game/round/[number]

Purpose: Fetch round scenario data

Auth: team-token

Response: Round variables (artist, venue, weather, pricing, etc.) without answers

GET /api/game/training-data

Purpose: Fetch all 60 historical training events

Auth: team-token

Response: Array of TrainingMatch objects (without actualGa / actualVip for active rounds)

POST /api/game/submit

Purpose: Submit team predictions

Auth: team-token

Body:

```
{
  "roundNumber": 1,
  "predictedGa": 32000,
  "predictedVip": 5500,
  "reasoning": "Based on the high artist prestige and weekend timing..."
}
```

Response: Submission result with score breakdown

POST /api/game/start-round

Purpose: Start the round timer for the team

Auth: team-token

Response: { started: true, timerSeconds: 600 }

GET /api/game/results

Purpose: Fetch team's past submission results

Auth: team-token

Response: Array of Submission objects with scores

GET /api/game/leaderboard

Purpose: Get ranked team standings

Response: [{ teamCode, name, totalScore, rank }]

GET /api/game/announcements

Purpose: Get active broadcast messages

Response: Array of Announcement objects

6.3 Admin Endpoints

GET/PUT /api/admin/game-control

Purpose: Read/update global game state

Auth: oxg-session (ADMIN)

PUT Body: { gameStatus, activeRound, timerEnabled, leaderboardEnabled }

GET/PUT /api/admin/scoring

Purpose: Read/update scoring configuration

Auth: oxg-session (ADMIN)

PUT Body: { gaWeight, vipWeight, maxRoundScore, penalties }

GET/POST /api/admin/teams

Purpose: List teams / Create new team

Auth: oxg-session (ADMIN)

POST /api/admin/bulk-import

Purpose: Import teams from Excel file

Auth: oxg-session (ADMIN)

Body: FormData with .xlsx file

GET /api/admin/rounds

Purpose: List all 6 game rounds with answers

Auth: oxg-session (ADMIN)

GET /api/admin/answers

Purpose: View correct answers for all rounds

Auth: oxg-session (ADMIN)

GET /api/admin/submissions

Purpose: View all team submissions across rounds

Auth: oxg-session (ADMIN)

POST /api/admin/announcement

Purpose: Create a new broadcast announcement

Auth: oxg-session (ADMIN)

GET /api/admin/game/export/teams

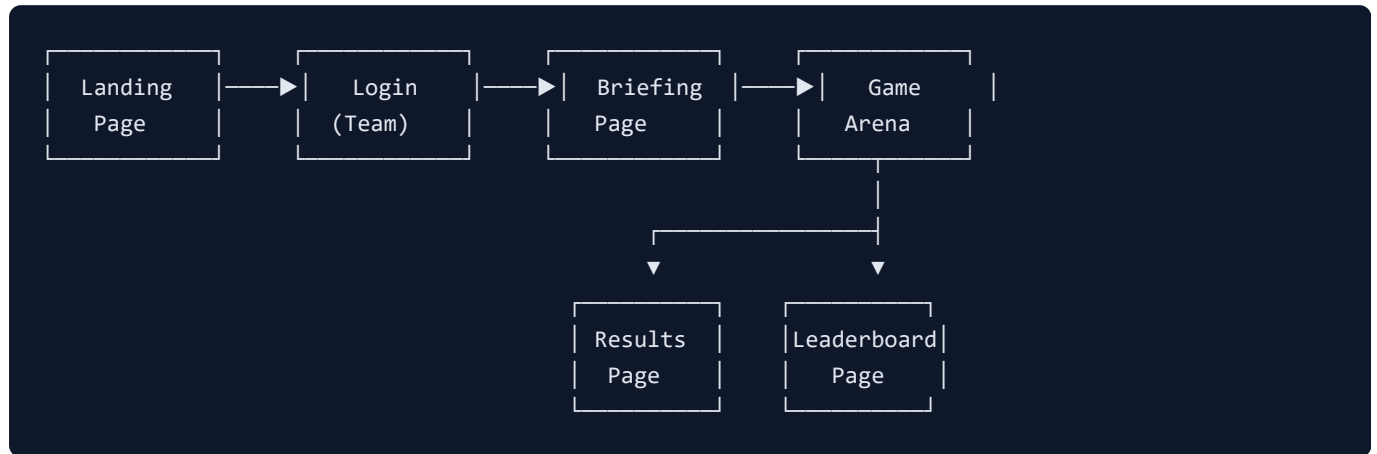
GET /api/admin/game/export/submissions

Purpose: Export data as Excel downloads

Auth: oxg-session (ADMIN)

7. Frontend Pages & User Flow

7.1 User Journey



7.2 Page Details

Landing Page (/)

- Hero section with game title "BEYOND THE STAGE"
- Badge: "UG Business Analytics Game 2026"
- Quick stats: 60 Events · 6 Rounds · 10m/Round · 1000 Max Pts
- Login form with Team Code + Access Code (with eye toggle)
- Session conflict handling with force login option
- Game completion "Thank You" screen
- Overview section: How It Works (4 steps)
- Variable dictionary section listing all 15+ demand drivers
- Footer with KSOM & Analytics Konclave branding

Team Login (/login)

- Standalone login card with Team Name + Access Code
- Eye toggle for PIN visibility
- Session conflict handling
- Link to Admin Portal

Admin Login (/login/admin)

- Admin branding with Shield icon
- Email + Password fields with eye toggle
- "ACCESS COMMAND CENTER" button

Game Briefing (/game/briefing)

- War Room Briefing with case study
- Tabbed interface: Case Study · Round Brief · Data & Resources
- Business scenario description
- Forecasting targets explanation
- Variable dictionary with 15+ demand drivers
- Data download buttons (Excel/CSV)
- Round scenario reveal with all input variables

Game Arena (/game)

- Active round display with countdown timer
- Round scenario card with all variables
- Dual input fields: GA Forecast (0–50,000) and VIP Forecast (0–10,000)
- Reasoning text area
- Submit button with confirmation modal
- Phase management: loading → briefing → prep → playing → review → confirm → result → transition → completed

Market Analysis (/game/market)

- Interactive charts via Recharts
- All 60 historical events displayed
- Filter and sort capabilities

Results (/game/results)

- Round-by-round score breakdown
- Accuracy percentages for GA and VIP
- Cumulative score tracking

Leaderboard (/game/leaderboard)

- Ranked team standings
 - Total score display
 - Real-time updates
-

8. Components Library

8.1 Shared Components

LogoHeader.tsx

- Displays KSOM logo + divider + Analytics Konclave logo
- Center title: "Beyond The Stage"
- Subtitle: "UG Business Analytics Game 2026"

Navbar.tsx

- Main navigation bar
- Responsive design
- Team info display

ThemeToggle.tsx

- Light/Dark mode switcher
- Persists preference

CaseStudyModal.tsx

- Full-screen modal overlay
- Contains complete game rules, lore, and variable explanations
- Scrollable with sections

8.2 Game Components

ResourcePanel.tsx

- Complex data visualization panel using Recharts
 - Renders 60 historical training events as interactive charts
 - Trend analysis for fuel, demand, fare, and occupancy
 - Used during gameplay for reference
-

9. Game Mechanics & Scoring

9.1 Game Structure

Phase	Description	Duration
Lobby	Teams register and wait	Until admin starts
Round 1–6	Each round presents a unique scenario	10 min (default)
Completed	All rounds finished, final rankings shown	—

9.2 Forecasting Variables (15 Demand Drivers)

#	Variable	Type	Description
1	Artist/Act	String	The performing artist or act
2	Venue Type	String	Arena, Stadium, Open-Air, etc.
3	Day of Week	String	Mon–Sun
4	Is Weekend	Binary (0/1)	Weekend flag
5	Is Holiday	Binary (0/1)	Holiday flag
6	Weather Condition	String	Clear, Rainy, Cloudy, etc.
7	Temperature	Float	Temperature reading
8	Promo Level	String	Low, Medium, High
9	Promo Score	Float	Promotional intensity index
10	Artist Prestige Index	Float	Artist fame/popularity rating
11	Competitor Events	Int	Number of competing events
12	Social Buzz	Float	Social media buzz index
13	GA Ticket Price	Float	General admission ticket price
14	VIP Ticket Price	Float	VIP/Premium ticket price
15	Historical Avg Occupancy	Float	Past average occupancy rate
16	Days Before Sold Out	Float	Sellout speed indicator

9.3 Forecasting Targets

Target	Range	Description
General Admission (GA)	0 – 50,000	Basic ticket occupancy
VIP / Premium	0 – 10,000	Premium ticket occupancy

9.4 Scoring Algorithm

The scoring engine is implemented in `src/lib/scoring.ts`:

Score Calculation:

1. GA Error = $|\text{predictedGA} - \text{actualGA}| / \text{actualGA} \times 100$
2. VIP Error = $|\text{predictedVIP} - \text{actualVIP}| / \text{actualVIP} \times 100$
3. GA Accuracy = $\max(0, 100 - \text{GA Error})$
4. VIP Accuracy = $\max(0, 100 - \text{VIP Error})$
5. Weighted Accuracy = $(\text{GA Accuracy} \times \text{gaWeight}) + (\text{VIP Accuracy} \times \text{vipWeight})$
Default weights: $\text{gaWeight} = 0.6$, $\text{vipWeight} = 0.4$
6. Penalty Adjustments:
 - Over-forecast: multiply error by $\text{overForecastPenalty}$ (default 1.0)
 - Under-forecast: multiply error by $\text{underForecastPenalty}$ (default 1.0)
7. Final Score = $(\text{Weighted Accuracy} / 100) \times \text{maxRoundScore}$
Default $\text{maxRoundScore} = 1000$

9.5 Timer Mechanism

- Timer is managed globally via `GameControl.timerEnabled` and per-team via `Team.roundStartedAt`
- When a team starts a round, `roundStartedAt` is set to the current timestamp
- Remaining time = `timerSeconds - elapsed seconds since roundStartedAt`
- Timer state is persisted in `localStorage` for same-device recovery
- Server is authoritative — client syncs from `/api/game/status`

10. Admin Dashboard

10.1 Dashboard Overview (/admin)

- Game status summary
- Active round indicator
- Total teams count
- Submission statistics

10.2 Admin Pages

Page	Path	Features
Overview	/admin	Dashboard stats, quick actions
Teams	/admin/teams	List all teams, view PINs (eye toggle), create/delete teams, generate random PINs
Bulk Import	/admin/teams/bulk-import	Upload Excel file to create multiple teams
Game Control	/admin/game-control	Set active round, enable/disable timer, toggle leaderboard, control game state
Rounds	/admin/rounds	View all 6 rounds with scenario details
Scoring	/admin/scoring	Configure weights, penalties, max scores
Submissions	/admin/submissions	View all team submissions across all rounds
Answers	/admin/answers	View/reveal correct answers for each round
Announcements	/admin/announcements	Create broadcast messages for teams
Leaderboard	/admin/leaderboard	Admin view of rankings

10.3 Team Management Features

- **Create Team:** Manual creation with custom team code and PIN
- **Generate PIN:** Auto-generate random secure PINs
- **Eye Toggle:** Show/hide PINs in the team table
- **Bulk Import:** Upload .xlsx file with team data

- **Export:** Download team data as Excel
 - **Session Management:** View active sessions, force logout
-

11. Types & Utilities

11.1 Core Types (`src/types/index.ts`)

```
interface RoundData {
  roundNumber: number;
  title: string;
  theme: string;
  instructions: string;
  // ... all round variables
}

interface SubmissionResult {
  predictedGa: number;
  predictedVip: number;
  gaError: number;
  vipError: number;
  accuracy: number;
  score: number;
}

interface TrainingMatch {
  eventId: string;
  // ... all 15+ variables
  actualGa: number;
  actualVip: number;
}
```

ROUND_THEMES Constant

Defines thematic scenarios for each of the 6 rounds.

HOW_IT_WORKS_STEPS Constant

Defines the 4-step onboarding flow shown on the landing page.

11.2 Utility Functions (`src/lib/`)

`auth.ts`

- `createToken(payload)` — Creates JWT via jose
- `verifyToken(token)` — Verifies JWT and returns payload
- `hashPassword(password)` — bcrypt hash (12 salt rounds)
- `comparePassword(input, hash)` — bcrypt compare

- `getTeamFromCookie()` — Extracts team info from cookie
- `getAdminFromCookie()` — Extracts admin info from cookie

`scoring.ts`

- `calculateScore(predicted, actual, config)` — Core scoring algorithm

`utils.ts`

- `cn(...classes)` — Tailwind class merger (clsx)
- `formatNumber(n)` — Number formatting with commas
- `formatTime(seconds)` — MM:SS timer display
- `getRelativeTime(date)` — "2 minutes ago" style formatting

`prisma.ts`

- Global Prisma client singleton (prevents connection exhaustion in dev)
-

12. Configuration & Environment

12.1 Environment Variables

Variable	Required	Description
DATABASE_URL	✓	PostgreSQL connection string (pooled)
DIRECT_URL	✓	Direct database connection (for migrations)
JWT_SECRET	✓	Secret key for JWT signing
NEXT_PUBLIC_APP_URL	✗	Public application URL
NODE_ENV	✗	Environment mode (development/production)

12.2 Build Configuration

next.config.mjs : - ESLint errors ignored during build (ignoreDuringBuilds: true) - TypeScript errors ignored during build (ignoreBuildErrors: true)

tsconfig.json : - Path alias: @/* → ./src/* - Strict mode enabled

13. Deployment Guide

13.1 Prerequisites

- Nodejs 18+
- PostgreSQL database (Neon recommended)
- npm or yarn

13.2 Local Development

```
# 1. Clone repository
git clone https://github.com/Avinash-Panda-gg/optix-ug-game.git

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
# Edit .env with your database credentials

# 4. Initialize database
npm run db:push    # Push schema to database
npm run db:seed    # Seed with training data + admin account

# 5. Start development server
npm run dev        # Runs on http://localhost:3000
```

13.3 Production Build

```
npm run build      # Runs: prisma generate && next build
npm start          # Starts production server
```

13.4 Vercel Deployment

1. Connect GitHub repo to Vercel
2. Set environment variables in Vercel dashboard
3. Build command: `npx prisma generate && next build`
4. After deploy: Run `npx prisma db push` and `npx prisma db seed`

13.5 Default Admin Credentials

Field	Value
Email	admin@housefull.com
Password	Admin@123

 **Change default credentials immediately after first login.**

14. Project Structure

```
opti-x-galaxy-ug/
├─ prisma/
│  └─ schema.prisma      # Database schema definition
│  └─ seed.ts            # Database seed script
├─ public/               # Static assets (logos, images)
├─ src/
│  └─ app/
│     └─ api/
│        └─ auth/
│           ├── login/route.ts
│           ├── team-login/route.ts
│           ├── logout/route.ts
│           ├── team-logout/route.ts
│           ├── register/route.ts
│           └─ me/route.ts
│        └─ game/
│           ├── status/route.ts
│           ├── round/[number]/route.ts
│           ├── training-data/route.ts
│           ├── submit/route.ts
│           ├── start-round/route.ts
│           ├── results/route.ts
│           ├── leaderboard/route.ts
│           └─ announcements/route.ts
│        └─ admin/
│           ├── teams/route.ts
│           ├── bulk-import/route.ts
│           ├── game-control/route.ts
│           ├── scoring/route.ts
│           ├── rounds/route.ts
│           ├── answers/route.ts
│           ├── submissions/route.ts
│           ├── announcement/route.ts
│           └─ game/export/
│              ├── teams/route.ts
│              └─ submissions/route.ts
│  └─ admin/
│     ├── layout.tsx
│     ├── page.tsx
│     ├── teams/
│     │  ├── page.tsx
│     │  └─ bulk-import/page.tsx
│     ├── game-control/page.tsx
│     ├── rounds/page.tsx
│     ├── scoring/page.tsx
│     ├── submissions/page.tsx
│     ├── answers/page.tsx
│     ├── announcements/page.tsx
│     └─ leaderboard/page.tsx
│  └─ game/
│     ├── layout.tsx
│     ├── page.tsx
│     └─ briefing/page.tsx
```

```
├── market/page.tsx
├── results/page.tsx
├── leaderboard/page.tsx
├── login/
│   ├── page.tsx
│   └── admin/page.tsx
├── register/page.tsx
├── case-study/page.tsx
├── guidelines/page.tsx
├── layout.tsx
├── page.tsx
├── components/
│   ├── shared/
│   │   ├── LogoHeader.tsx
│   │   ├── Navbar.tsx
│   │   ├── ThemeToggle.tsx
│   │   └── CaseStudyModal.tsx
│   └── game/
│       └── ResourcePanel.tsx
├── lib/
│   ├── auth.ts
│   ├── scoring.ts
│   ├── utils.ts
│   └── prisma.ts
├── types/
│   └── index.ts
├── middleware.ts
├── .env.example
├── next.config.mjs
├── tailwind.config.ts
├── tsconfig.json
├── package.json
└── README.md
```